Real-world (rebuild Doc04's extraction, LCEL-style) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The mapping, and the two versions
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The mapping, and the two versions

Basic Version

You already built a structured-extraction feature in Doc04's Build Task: `get_structured_reply(client, text)` in `chat_client.py`, returning an `ExtractedInfo` from `schemas.py`. This exercise writes the **same** feature a second way, with LCEL, and keeps both.

First copy `chat_client.py` and `schemas.py` from `04_openai_api/practice/build_task/` into `05_langchain_fundamentals/practice/`, unchanged — the same "copy, don't rewrite" rule as Doc01's `config.py`.

Then map each raw step to an LCEL piece: the `messages` list → a prompt template; the API call → `ChatOpenAI`; `response_format=ExtractedInfo` → `.with_structured_output(ExtractedInfo)`.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Doc04's raw call sends **two** messages: a system message (`EXTRACT_PROMPT`) and the user's text. `from_template(...)` builds only one message, so use `ChatPromptTemplate.from_messages([("system", EXTRACT_PROMPT), ("human", "{text}")])` — import `EXTRACT_PROMPT` from `chat_client.py` so both versions get the same words.

What does "match" mean for two model calls? A **name** has one right answer, so both versions must agree on it. **Intent** is free wording ("reset her password" vs. "reset password") that can differ between any two calls — even two raw ones. Compare `raw_result.name == lcel_result.name`.

Then split the work: `run_comparison(test_inputs) -> list[ComparisonRow]` computes, `main()` reports. That's the shape the Build Task copies.

Difference between Basic and Intermediate: Basic names the mapping. Intermediate adds the two-message prompt, a fair definition of "match", and the compute/report split.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
import EXTRACT_PROMPT, create_client, get_structured_reply, ExtractedInfo
prompt = system message EXTRACT_PROMPT + human message {text}
lcel_chain = prompt | model.with_structured_output(ExtractedInfo)
for each of 3 test inputs:
    raw = get_structured_reply(client, text)
    lcel = lcel_chain.invoke({"text": text})
    print both
```

```
# lcel_vs_raw_sdk_practice.py
prompt = ChatPromptTemplate.from_messages(
    [("system", EXTRACT_PROMPT), ("human", "{text}")]
)
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
lcel_chain = prompt | model.with_structured_output(ExtractedInfo)
```

Add the imports, `client = create_client()`, the 3 inputs and the loop. Use inputs that each contain a name — Doc04's `get_structured_reply()` returns `None` when it finds nothing.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
@dataclass
class ComparisonRow: input_text, raw_result, lcel_result, matched

function run_comparison(test_inputs) -> list[ComparisonRow]:
    client, lcel_chain
    for each text:
        run both, matched = same name
        add a ComparisonRow
    return the rows

main: run_comparison, count matched, print any mismatch
```

Write it, run it, then compare against the Solution.

Difference between Basic and Intermediate: Basic prints both results for your eyes. Intermediate computes the match itself and returns typed rows, so the Build Task can reuse the logic unchanged.

Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution